```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error , mean_squared_error , r2_score
```

```
df = pd.read_csv("/content/car data.csv")
df
```

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0
...	...	...	...	...	...	...	...	...	...
296	city	2016	9.50	11.60	33988	Diesel	Dealer	Manual	0
297	brio	2015	4.00	5.90	60000	Petrol	Dealer	Manual	0
298	city	2009	3.35	11.00	87934	Petrol	Dealer	Manual	0
299	city	2017	11.50	12.50	9000	Diesel	Dealer	Manual	0
300	brio	2016	5.30	5.90	5464	Petrol	Dealer	Manual	0

301 rows × 9 columns

Next steps: Generate code with df | New interactive sheet

```
#info
print ("df informaton")
print(df.info())
```

```
df informaton
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 301 entries, 0 to 300
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Car_Name       301 non-null    object
 1   Year           301 non-null    int64
 2   Selling_Price  301 non-null    float64
 3   Present_Price  301 non-null    float64
 4   Kms_Driven     301 non-null    int64
 5   Fuel_Type      301 non-null    object
 6   Seller_Type    301 non-null    object
 7   Transmission   301 non-null    object
 8   Owner          301 non-null    int64
dtypes: float64(2), int64(3), object(4)
memory usage: 21.3+ KB
None
```

```
#define independent variables
x=df[['Present_Price','Kms_Driven','Year']]
```

```
#define dependent variable
y=df['Selling_Price']
```

```
from sklearn.model_selection import train_test_split
#splite dataset tranning and testing
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

```
#tranning data
print(x_train)
```

```
     Present_Price  Kms_Driven  Year
184          0.750       26000  2008
132          0.950        3500  2017
```

```
194          0.787       50000  2008
75           6.800       36000  2015
111          1.500        8700  2016
..             ...         ...   ...
188          0.570       18000  2013
71          12.480       45000  2011
106          3.450       16500  2014
270         10.000       69341  2011
102          1.780        4000  2017

[240 rows x 3 columns]
```

```
#testing
print(y_test)
```

```
177     0.35
289    10.11
228     4.95
198     0.15
60      6.95
       ...
234     5.50
296     9.50
281     2.10
285     7.40
182     0.30
Name: Selling_Price, Length: 61, dtype: float64
```

```
from sklearn.linear_model import LinearRegression
#create multiple linear regression
model = LinearRegression()
```

```
df.head()
```

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0

```
df.shape
```

```
(301, 9)
```

```
df.isnull()
```

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
296	False	False	False	False	False	False	False	False	False
297	False	False	False	False	False	False	False	False	False
298	False	False	False	False	False	False	False	False	False
299	False	False	False	False	False	False	False	False	False
300	False	False	False	False	False	False	False	False	False

301 rows × 9 columns

```
df.isnull().sum()
```

	0
Car_Name	0
Year	0
Selling_Price	0
Present_Price	0
Kms_Driven	0
Fuel_Type	0
Seller_Type	0
Transmission	0
Owner	0

dtype: int64

```
df.duplicated().sum()
```

```
np.int64(2)
```

```
df = df.drop_duplicates()
df
```

	Car_Name	Year	Selling_Price	Present_Price	Kms_Driven	Fuel_Type	Seller_Type	Transmission	Owner
0	ritz	2014	3.35	5.59	27000	Petrol	Dealer	Manual	0
1	sx4	2013	4.75	9.54	43000	Diesel	Dealer	Manual	0
2	ciaz	2017	7.25	9.85	6900	Petrol	Dealer	Manual	0
3	wagon r	2011	2.85	4.15	5200	Petrol	Dealer	Manual	0
4	swift	2014	4.60	6.87	42450	Diesel	Dealer	Manual	0
...	...	...	...	...	...	...	...	...	...
296	city	2016	9.50	11.60	33988	Diesel	Dealer	Manual	0
297	brio	2015	4.00	5.90	60000	Petrol	Dealer	Manual	0
298	city	2009	3.35	11.00	87934	Petrol	Dealer	Manual	0
299	city	2017	11.50	12.50	9000	Diesel	Dealer	Manual	0
300	brio	2016	5.30	5.90	5464	Petrol	Dealer	Manual	0

299 rows × 9 columns

```
print ("\n duplicate values")
print(df.duplicated().sum())
```

```
 duplicate values
0
```

```
le = LabelEncoder()
```

```
df.loc[:,"Fuel_Type"]= le.fit_transform(df.loc[:,"Fuel_Type"])
df.loc[:,"Transmission"]= le.fit_transform(df.loc[:,"Transmission"])
df.loc[:,"Seller_Type"]= le.fit_transform(df.loc[:,"Seller_Type"])
```

```
print(df.head())
```

```
  Car_Name  Year  Selling_Price  Present_Price  Kms_Driven Fuel_Type  \
0     ritz  2014           3.35           5.59       27000         2
1      sx4  2013           4.75           9.54       43000         1
2     ciaz  2017           7.25           9.85        6900         2
3  wagon r  2011           2.85           4.15        5200         2
4    swift  2014           4.60           6.87       42450         1

  Seller_Type Transmission  Owner
0           0            1      0
```

```
1           0           1      0
2           0           1      0
3           0           1      0
4           0           1      0
```

```
X= df[["Year","Present_Price","Kms_Driven","Fuel_Type","Seller_Type","Transmission","Owner"]]
Y = df["Selling_Price"]
```

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.20,random_state=42)
```

```
model = LinearRegression()
model.fit(x_train,y_train)
```

▾ LinearRegression

LinearRegression()

```
Y_pred = model.predict(x_test)
print ("Predicated Selling Price:")
print(Y_pred)
```

```
Predicated Selling Price:
[ 2.10830482  8.83407476  4.85685547 -0.11971186 10.06284876  5.95448779
  2.12673808  1.39209688  2.10030038  6.03945488  7.78547159  1.2335922
  7.13907998  2.53827219  5.31200565  2.97284635  0.94998598 11.87750733
  2.54829042  3.21341152  0.86783645  6.6629614   4.87749903  2.16967852
  1.43611483  3.53022813  5.56099373  2.52363397  3.02585073  2.59213024
  0.89423259  8.11352303 -0.36184397  1.92277873  7.36183311  4.52639347
  8.01334065  8.27059098  2.73114769  8.68742349  3.95428148  2.76962218
  4.22906875  1.33065794  5.80137072  1.09517182  7.92441135 10.87624478
  2.84912711  3.51745028  5.3185706   3.03099707 19.88827216 15.35196415
  6.18771287  8.81932712  4.32305064  7.78605337  1.27242417  6.20233706
  0.77709663]
```

```
mae = mean_absolute_error(y_test,Y_pred)
mse = mean_squared_error(y_test,Y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test,Y_pred)
```

```
print ("Mean Absolute Error:",mae)
print ("Mean Squared Error:",mse)
print ("R2 Score:",r2)
```

```
Mean Absolute Error: 1.3638305462396507
Mean Squared Error: 4.010554030275675
R2 Score: 0.825897448934144
```